МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»
Факультет інформатики та обчислювальної техніки

Лабораторна робота №1

з дисципліни «Програмування мовою Java»

Тема: «Робота з циклами, масивами та рядками в Java»

Виконав: студент III курсу, групи ІМ-44
Дудник Олександр Михайлович
Номер у загальному списку: 27
Варіант: 7

Київ – 2026

1. Мета роботи

Здобути досвід роботи з Java Development Kit та обраним інтегрованим середовищем розробки при практичному вирішенні обраного завдання, ознайомитись з процесом виконання Java програм.

2. Індивідуальне завдання (Варіант 7)

Відповідно до номера **27** у загальному списку слухачів дисципліни (за правилом циклічного розподілу з методичних вказівок: 11-й у списку бере 1-й варіант, 12-й — 2-й, ..., 21-й — 1-й, 27-й номер бере **7-й варіант**) створити консольний додаток:

«Серед простих чисел, які не перевищують заданий n, знайти таке, в двійковій формі якого максимальна кількість нулів. Просте число — це натуральне число, яке ділиться на 1 та на себе.»

3. Опис алгоритму та структури програми

Програма реалізована у вигляді консольного застосунку мовою Java (OpenJDK 21) у середовищі розробки Visual Studio Code. Згідно з вимогами теми лабораторної роботи («Робота з циклами, масивами та рядками в Java») та з урахуванням примітки щодо лабораторної роботи №7, алгоритм побудовано виключно на базових циклах, примітивних масивах і рядках без використання колекцій або Stream API.

Структура програми включає такі методи:

1. **Метод sieveOfEratosthenes(int max):** реалізує алгоритм «Решето Ератосфена» на базі булевого масиву boolean[] isPrime для пошуку всіх простих чисел у діапазоні від 2 до n включно. Часова складність

становить O(n log log n), просторова — O(n). Для запобігання цілочисельному переповненню 32-бітного типу int умову зовнішнього циклу записано у безпечній формі p <= max / p.

2. **Метод countSignificantZeros(int x):** обчислює кількість значущих нулів у двійковому поданні натурального числа x (ведучі нулі ліворуч від найстаршої одиниці у 32-бітному типі int не враховуються). Аналіз виконується у циклі while (x > 0) за допомогою побітової кон'юнкції (x & 1) == 0 та беззнакового побітового зсуву праворуч x >>>= 1 за O(log2(x)) без додаткових алокацій пам'яті.
3. **Метод toBinaryStringCustom(int x):** формує рядкове двійкове подання числа за допомогою символьного масиву char[32] та побітових операцій для наочного виведення результатів.
4. **Метод main(String[] args):** забезпечує введення та валідацію натурального числа n (n >= 2), знаходить максимальну кількість значущих нулів серед простих чисел діапазону, фіксує перше (найменше) та останнє (найбільше) таке просте число, формує результуючий масив int[] allBestPrimes і виводить результати та час виконання у мікросекундах.

4. Вихідний код програми (Lab1Variant7.java)

```
import java.io.PrintStream;
import java.nio.charset.StandardCharsets;
import java.util.Scanner;

public class Lab1Variant7 {

    public static void main(String[] args) {
```

```
        System.setOut(new PrintStream(System.out, true,
StandardCharsets.UTF_8));

        try (Scanner scanner = new Scanner(System.in,
StandardCharsets.UTF_8)) {
            System.out.print("Введіть натуральне число n (n >= 2): ");

            if (!scanner.hasNextInt()) {
                System.err.println("Помилка: введене значення не є цілим
числом типу int.");
                return;
            }

            int n = scanner.nextInt();
            if (n < 2) {
                System.err.println("Помилка: найменше просте число — 2.
Введіть n >= 2.");
                return;
            }

            long startTime = System.nanoTime();

            boolean[] isPrime = sieveOfEratosthenes(n);

            int maxZeros = -1;
            int firstBestPrime = -1;
            int lastBestPrime = -1;
            int winnersCount = 0;

            for (int i = 2; i <= n; i++) {
                if (isPrime[i]) {
                    int zeros = countSignificantZeros(i);
                    if (zeros > maxZeros) {
                        maxZeros = zeros;
                        firstBestPrime = i;
                        lastBestPrime = i;
                        winnersCount = 1;
```

```
                    } else if (zeros == maxZeros) {
                        lastBestPrime = i;
                        winnersCount++;
                    }
                }
            }

            int[] allBestPrimes = new int[winnersCount];
            int idx = 0;
            for (int i = 2; i <= n; i++) {
                if (isPrime[i] && countSignificantZeros(i) == maxZeros) {
                    allBestPrimes[idx++] = i;
                }
            }

            long elapsedMicros = (System.nanoTime() - startTime) / 1000;

            System.out.println("Верхня межа діапазону (n): " + n);
            System.out.println("Максимальна кількість нулів у двійковій
формі: " + maxZeros);
            System.out.println("Шукане просте число (перше знайдене): " +
firstBestPrime
                    + " -> " + toBinaryStringCustom(firstBestPrime) +
"(2)");

            if (winnersCount > 1) {
                System.out.println("Шукане просте число (найбільше в
діапазоні): " + lastBestPrime
                        + " -> " + toBinaryStringCustom(lastBestPrime) +
"(2)");
                System.out.print("Усі прості числа з " + maxZeros + "
нулями (" + winnersCount + " шт.): ");
                for (int i = 0; i < allBestPrimes.length; i++) {
                    System.out.print(allBestPrimes[i] + " [" +
toBinaryStringCustom(allBestPrimes[i]) + "]");
                    if (i < allBestPrimes.length - 1) {
                        System.out.print(", ");
```

```
                    }
                }
                System.out.println();
            }
            System.out.println("Час обчислення: " + elapsedMicros + "
мкс");
        }
    }

    public static int countSignificantZeros(int x) {
        int zeros = 0;
        while (x > 0) {
            if ((x & 1) == 0) {
                zeros++;
            }
            x >>>= 1;
        }
        return zeros;
    }

    public static boolean[] sieveOfEratosthenes(int max) {
        boolean[] isPrime = new boolean[max + 1];
        for (int i = 2; i <= max; i++) {
            isPrime[i] = true;
        }
        for (int p = 2; p <= max / p; p++) {
            if (isPrime[p]) {
                for (int multiple = p * p; multiple <= max; multiple += p)
{
                    isPrime[multiple] = false;
                }
            }
        }
        return isPrime;
    }

    public static String toBinaryStringCustom(int x) {
```

```
        if (x == 0) {
            return "0";
        }
        char[] buffer = new char[32];
        int pos = 32;
        while (x > 0) {
            buffer[--pos] = (char) ('0' + (x & 1));
            x >>>= 1;
        }
        return new String(buffer, pos, 32 - pos);
    }
}
```

5. Результати тестування програми

Тест 1 (n = 10):

```
PS C:\Users\catc6\Desktop\Un\Java> java Lab1Variant7.java
Введіть натуральне число n (n >= 2): 10
Верхня межа діапазону (n): 10
Максимальна кількість нулів у двійковій формі: 1
Шукане просте число (перше знайдене): 2 -> 10(2)
Шукане просте число (найбільше в діапазоні): 5 -> 101(2)
Усі прості числа з 1 нулями (2 шт.): 2 [10], 5 [101]
Час обчислення: 11 мкс
```

Тест 2 (n = 20):

```
PS C:\Users\catc6\Desktop\Un\Java> java Lab1Variant7.java
Введіть натуральне число n (n >= 2): 20
Верхня межа діапазону (n): 20
Максимальна кількість нулів у двійковій формі: 3
Шукане просте число (перше знайдене): 17 -> 10001(2)
Час обчислення: 8 мкс
```

Тест 3 (n = 100):

```
PS C:\Users\catc6\Desktop\Un\Java> java Lab1Variant7.java
Введіть натуральне число n (n >= 2): 100
Верхня межа діапазону (n): 100
Максимальна кількість нулів у двійковій формі: 4
Шукане просте число (перше знайдене): 67 -> 1000011(2)
Шукане просте число (найбільше в діапазоні): 97 -> 1100001(2)
Усі прості числа з 4 нулями (3 шт.): 67 [1000011], 73 [1001001], 97 [1100001]
Час обчислення: 17 мкс
```

6. Посилання на репозиторій GitHub

7. Висновки

Під час виконання лабораторної роботи №1 було здобуто практичний досвід роботи з Java Development Kit (OpenJDK 21) та інтегрованим середовищем розробки Visual Studio Code, а також досліджено процес компіляції і виконання консольних Java-програм. Відповідно до індивідуального варіанту №7 розроблено консольний додаток для пошуку простого числа з максимальною кількістю нулів у двійковій формі в діапазоні до заданого n із використанням базових циклів, примітивних масивів, рядків та побітових операцій. Проведене тестування на контрольних наборах даних (n = 10, n = 20, n = 100) підтвердило коректність та високу швидкодію розробленої програми.